

A Supplementary Material

Here, we present additional experimental results with DySurface that are not included in the main paper. We provide detailed descriptions of DySurface and several additional experimental results.

B Additional Implementation Details

B.1 Network Architecture

B.1.1 Geometry Representation (SDF).

To represent the continuous implicit surface, we utilize a multi-resolution hash grid architecture. The spatial coordinates are encoded using a 16-level progressive band hash grid with a base resolution of 32, 2 feature dimensions per level, and a maximum hash map size of 2^{21} . The Geometry MLP consists of 1 hidden layer with 128 neurons and employs Softplus activation along with weight normalization. To ensure stable early training, we apply geometric initialization to approximate an initial sphere with a radius of 0.8. The network outputs a 128-dimensional geometric feature alongside the SDF value. The color network predicts RGB values using the 128-dimensional geometric feature, time embeddings, and viewing directions encoded via Spherical Harmonics (degree of 4). The color network is composed of 2 hidden layers with 128 neurons each, utilizing ReLU activations for intermediate layers and a final Sigmoid activation to bound the RGB output.

B.1.2 Sparse Voxel & Dynamic Deformation.

We anchor the Gaussian features into a sparse volumetric representation. The canonical space is bounded by a sparse voxel grid configured with a maximum resolution of 256^3 and a voxel size of 0.002. Each valid voxel stores a 6-dimensional latent feature vector. We apply an occupancy threshold of 0.001 to aggressively prune empty grid cells and maintain computational efficiency. Instead of heuristic functional bases, the non-rigid dynamics are modeled through a dedicated Delta network. The continuous deformation from the canonical to the dynamic space is parameterized by mapping the sparse voxel features through this Delta network, ensuring structurally coherent motion tracking.

Table 6: Loss weight configurations across the three progressive training stages of DySurface. Symbols correspond to those defined in the main paper. RGB rendering losses ($\mathcal{L}_{rgb}^{sdf}$, $\mathcal{L}_{rgb}^{mesh}$) use a weight of 1.0 by default in all stages.

Stage	Symbol	Loss Term	Value
GS Branch	λ_m	$\mathcal{L}_{mask}$	0.3
	λ_{reg}^{gs}	$\mathcal{L}_{reg}^{gs}$	1.0
VoxGS-DSDF Branch	λ_c	$\mathcal{L}_{cycle}$	1.0
	λ_{SG}	$\mathcal{L}_{SDF-GS}$	1.0
	λ_{dist}	$\mathcal{L}_{distill}$	1.0
	λ_e	$\mathcal{L}_{eik}$	0.1
	λ_s	$\mathcal{L}_{smooth}$	1.0
Mesh Refinement	λ_m^{mesh}	$\mathcal{L}_{mask}^{mesh}$	0.5
	λ_{lap}	$\mathcal{L}_{lap}$	1.0
	λ_{reg}^{mesh}	$\mathcal{L}_{reg}^{mesh}$	0.1

For volume rendering, we dynamically sample up to 1024 points per ray, which includes 256 fine samples to capture detailed topological structures accurately. Its dynamic field motion is learned using the same network structure as the Gaussian Splatting branch.

B.2 Hyperparameters and Training Details

Our framework is trained progressively in three stages using the AdamW optimizer. Each prior stage is frozen during the next stage optimization. The Gaussian Splatting branch is trained for 30,000 steps, followed by 30,000 steps for the VoxGS-DSDF branch, and a final 1,000 steps for the Mesh Refinement branch. The base learning rate is set to 0.01 for the texture network, while the geometry and deformation modules use 0.001. A sequential learning rate scheduler is applied, multiplying the learning rate by 0.1 at 20,000 steps, with additional decay at later milestones.

Due to space limitations in the main paper, we provide the complete loss weight configurations across all three training stages in Table 6. The reported values are fixed across all evaluated scenes, and the

majority of weights are simply set to 1.0; per-scene configurations, when adjusted in specific scenes, will be released in our codebase.

In Eq. (8), ϵ serves as a spatial distance threshold that defines the boundary proximity for anchoring the implicit zero-level set to the explicit canonical Gaussian. We set ϵ relative to the overall spatial scale of the scene. For the D-NeRF benchmark, we set $\epsilon = 1 \times 10^{-4}$, which corresponds to approximately 0.008 of the object’s bounding radius. Setting ϵ too large relaxes the geometric anchoring constraint, allowing the implicit SDF field to drift away from explicit Gaussians and causing loss of sharp surface details. Conversely, setting ϵ too small overly constrains the continuous SDF field to discrete Gaussian centers, which can introduce local surface noise. Our relative scaling choice balances smooth zero-level set continuity with sharp explicit boundary alignment.

B.3 Details of Temporal Smoothness Term

To encourage temporally smooth deformation in the VoxGS-DSDf branch, we regularize the predicted backward deformation offsets of ray-sampled query points. Let $\Delta \mathbf{q}_i(t)$ denote the predicted deformation offset of the i -th query point at time t . Given its neighboring timestamps t_{prev} and t_{next} , we randomly sample intermediate timestamps as

$$\tau_{\text{prev}} = t + \alpha(t_{\text{prev}} - t), \quad \alpha \sim \mathcal{U}(0, 1), \quad (11)$$

and

$$\tau_{\text{next}} = t + \beta(t_{\text{next}} - t), \quad \beta \sim \mathcal{U}(0, 1). \quad (12)$$

The temporal smoothness loss is defined as

$$\mathcal{L}_{\text{smooth}} = \mathcal{L}_{\text{smooth}}^{\text{prev}} + \mathcal{L}_{\text{smooth}}^{\text{next}}, \quad (13)$$

where

$$\mathcal{L}_{\text{smooth}}^{\text{prev}} = \frac{1}{N} \sum_{i=1}^N \left\| \Delta \mathbf{q}_i(t) - \overline{\Delta \mathbf{q}_i(\tau_{\text{prev}})} \right\|_2^2, \quad (14)$$

and

$$\mathcal{L}_{\text{smooth}}^{\text{next}} = \frac{1}{N} \sum_{i=1}^N \left\| \Delta \mathbf{q}_i(t) - \overline{\Delta \mathbf{q}_i(\tau_{\text{next}})} \right\|_2^2. \quad (15)$$

Here, N denotes the number of ray-sampled query points, and the overline denotes a gradient-detach operation. The deformation offsets predicted at the intermediate timestamps are treated as detached temporal targets, encouraging the backward deformation field to vary smoothly over time.

C Additional Experiments

C.1 Evaluation Across Time Steps

Unlike prior works such as GaGS [S8], DG-Mesh [S7], 4DGS [S6], and FDNerF [S5], which only quantitatively measure novel view synthesis photometric quality, we are the first to explicitly measure and report the 3D geometry accuracy on the D-NeRF [S2] dataset.

We present additional quantitative results to provide a fully transparent and granular breakdown of our framework’s performance. Tab. 7 details the geometric evaluation scores (vIoU and Chamfer Distance) grouped by distinct continuous time steps ($t \in \{0.0, 0.5, 1.0\}$). Within each time step, we provide the metric for individual scenes alongside their cross-scene averages to demonstrate temporal stability. Furthermore, the overall photometric metrics (PSNR, SSIM, and LPIPS) are reported at the bottom of the table. This discrete time step breakdown confirms that our method maintains robust reconstruction fidelity and structural coherence not only near the canonical space but also at distant temporal states.

C.2 Results on Remaining Scenes

In Fig. 9, Fig. 12, and Fig. 13, we provide visual results on the remaining scenes of the D-NeRF [S2] dataset. The qualitative results demonstrate that DySurface consistently captures continuous deformation, high-fidelity rendering results, and preserves sharp structural details.

Table 7: **Comprehensive Detailed Quantitative Evaluation by Time Step.** We report the detailed metrics grouped by each specific time step ($t \in \{0.0, 0.5, 1.0\}$). Methods in parentheses indicate the mesh extraction method. HW stands for HellWarrior.

Metric	TiNeuVox	4DGS + (3DGS-to-PC)	GaGS + (GS2Mesh)	GaGS + (3DGS-to-PC)	DG-Mesh	D-2DGS	Ours
$t = 0.0$							
vIoU (Lego) $\uparrow$	0.1300	0.2252	0.3286	0.2446	0.2614	0.2196	0.3012
vIoU (T-Rex) $\uparrow$	0.5545	0.2764	0.0783	0.2878	0.5693	0.4949	0.5532
vIoU (HW) $\uparrow$	0.1991	0.2454	0.1990	0.2347	0.3037	0.3047	0.2906
vIoU (Avg) $\uparrow$	0.2945	0.2490	0.2020	0.2557	<u>0.3781</u>	0.3397	0.3817
CD (Lego) $\downarrow$	0.0180	0.0188	0.0170	0.0157	0.0242	0.0128	0.0132
CD (T-Rex) $\downarrow$	0.0073	0.0700	0.0211	0.0068	0.0046	0.0056	0.0051
CD (HW) $\downarrow$	0.0272	0.0232	0.0434	0.0230	0.0201	0.0135	0.0129
CD (Avg) $\downarrow$	0.0175	0.0373	0.0272	0.0150	0.0163	<u>0.0106</u>	0.0104
$t = 0.5$							
vIoU (Lego) $\uparrow$	0.1080	0.2210	0.2772	0.2289	0.2701	0.2200	0.3015
vIoU (T-Rex) $\uparrow$	0.1634	0.2542	0.0517	0.2198	0.5779	0.5029	0.6285
vIoU (HW) $\uparrow$	0.3122	0.2168	0.1566	0.2586	0.3164	0.3162	0.3101
vIoU (Avg) $\uparrow$	0.1945	0.2307	0.1618	0.2358	<u>0.3881</u>	0.3464	0.4134
CD (Lego) $\downarrow$	0.0174	0.0187	0.0171	0.0164	0.0242	0.0139	0.0139
CD (T-Rex) $\downarrow$	0.0068	0.0680	0.0275	0.0071	0.0042	0.0053	0.0048
CD (HW) $\downarrow$	0.0226	0.0216	0.0472	0.0185	0.0189	0.0121	0.0115
CD (Avg) $\downarrow$	0.0156	0.0361	0.0306	0.0140	0.0158	<u>0.0104</u>	0.0101
$t = 1.0$							
vIoU (Lego) $\uparrow$	0.0860	0.2001	0.2699	0.2191	0.2649	0.2019	0.2938
vIoU (T-Rex) $\uparrow$	0.5518	0.2799	0.0620	0.2513	0.5735	0.5014	0.5532
vIoU (HW) $\uparrow$	0.2292	0.2266	0.1934	0.2347	0.3002	0.3029	0.2841
vIoU (Avg) $\uparrow$	0.2890	0.2355	0.1751	0.2350	0.3795	0.3354	<u>0.3770</u>
CD (Lego) $\downarrow$	0.0168	0.0166	0.0160	0.0145	0.0225	0.0139	0.0129
CD (T-Rex) $\downarrow$	0.0072	0.0750	0.0492	0.0069	0.0045	0.0055	0.0052
CD (HW) $\downarrow$	0.0260	0.0215	0.0403	0.0230	0.0206	0.0139	0.0128
CD (Avg) $\downarrow$	0.0167	0.0377	0.0352	0.0148	0.0158	<u>0.0111</u>	0.0103
Overall Metrics							
PSNR (Avg) $\uparrow$	28.71	28.97	32.24		25.23	25.82	<u>31.09</u>
SSIM (Avg) $\uparrow$	0.9554	0.9678	0.9754		0.9187	0.9374	<u>0.9735</u>
LPIPS (Avg) $\downarrow$	0.0542	0.0261	<u>0.0182</u>		0.1027	0.0681	0.0161

We report the quantitative photometric metrics on the remaining dynamic scenes in Tab. 8 and Fig. 8. We compare against representative baselines, including 3D-GS [S1], D-NeRF [S2], TiNeuVox [S3], NDVG [16], FDNerF [S5], 4D-GS [S6], DG-Mesh [S7], D-2DGS [53], and GaGS [S8]. As our framework is fundamentally designed to extract geometrically accurate and topologically coherent surfaces, it enforces spatial regularization of the continuous SDF field. Consequently, while purely NVS-dedicated methods may yield marginally higher photometric performance by freely scattering unconstrained Gaussians to overfit the training views, they fail to recover meaningful geometry. In contrast, DySurface maintains highly competitive photometric fidelity while providing state-of-the-art geometric reconstruction, achieving an optimal balance between visual rendering quality and structural 3D geometry. Fig. 8 also shows that our method achieves a competitive rendering score against NVS-optimized methods, compared to the prior surface reconstruction methods [53, 26].

C.3 Results on Real-world Dataset

We additionally test our method on real-world datasets to validate practical applicability. In addition to our quantitative evaluations on the real-world Nerfies [32] dataset (Table 2 and Figure 5), we conducted additional experiments on the real-world DyCheck [S11] benchmark across 7 different scenes (*haru-sit*, *backpack*, *teddybear*, *pillow*, *creeper*, *mochi-high-five*, *sriracha-tree*). As presented

Table 8: **Quantitative comparison on the D-NeRF [S2] dataset.** We compare our method against state-of-the-art baselines. The best results are highlighted in **bold**.

Method	Hell Warrior			Mutant			Hook			Bouncing Balls		
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
3D-GS	15.39	0.8776	0.1300	21.75	0.9359	0.0575	18.69	0.8733	0.1144	22.56	0.9485	0.0647
D-NeRF	25.02	0.9506	0.0691	31.29	0.9739	0.0268	29.26	0.9650	0.1174	38.93	0.9900	0.1031
TiNeuVox	28.20	0.9661	0.0631	33.90	0.9771	0.0301	31.79	0.9718	0.0436	40.85	0.9913	0.0401
NDVG	26.49	0.9600	0.0670	34.41	0.9801	0.0270	30.00	0.9626	0.0463	37.52	0.9874	0.0751
FDNeRF	27.71	0.9665	0.0508	34.97	0.9810	0.0312	32.29	0.9756	0.0388	40.02	0.9912	0.0395
4D-GS	28.12	0.9730	0.0276	38.34	0.9936	0.0062	33.16	0.9810	0.0168	40.74	0.9941	0.0105
GaGS	32.27	0.9835	0.0164	41.43	0.9969	0.0029	36.96	0.9916	0.0076	43.59	0.9960	0.0061
DG-Mesh	25.46	0.9590	0.0842	30.40	0.9680	0.0550	27.88	0.9540	0.0740	29.14	0.9690	0.0990
D-2DGS	25.50	0.9583	0.1121	28.12	0.9600	0.0423	27.80	0.9620	0.0419	27.78	0.9690	0.0730
Ours	31.14	0.9835	0.0169	32.25	0.9833	0.0099	32.00	0.9822	0.0148	35.99	0.9856	0.0136

Method	Lego			T-Rex			Stand Up			Jumping Jacks		
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
3D-GS	23.10	0.9329	0.0567	25.75	0.9567	0.0474	19.38	0.9200	0.0909	20.72	0.9227	0.0980
D-NeRF	21.64	0.8394	0.1654	31.75	0.9767	0.0396	32.80	0.9818	0.0215	32.80	0.9810	0.0373
TiNeuVox	25.17	0.9217	0.0689	32.77	0.9783	0.0307	36.20	0.9859	0.0199	34.74	0.9823	0.0328
NDVG	25.04	0.9395	0.0534	32.62	0.9781	0.0330	33.22	0.9793	0.0302	31.25	0.9737	0.0398
FDNeRF	25.27	0.9390	0.0460	30.71	0.9731	0.0368	36.91	0.9878	0.0188	33.55	0.9812	0.0329
4D-GS	25.40	0.9434	0.0377	33.39	0.9869	0.0130	38.26	0.9923	0.0071	35.67	0.9882	0.0159
GaGS	25.44	0.9474	0.0329	39.03	0.9952	0.0052	42.21	0.9966	0.0028	37.96	0.9928	0.0088
DG-Mesh	21.29	0.8380	0.1590	28.95	0.9590	0.0650	30.21	0.9740	0.0510	31.77	0.9770	0.0450
D-2DGS	23.29	0.8872	0.1124	28.67	0.9670	0.0434	29.51	0.9768	0.0280	29.29	0.9741	0.0322
Ours	25.40	0.9439	0.0245	36.75	0.9932	0.0069	36.08	0.9903	0.0089	32.35	0.9822	0.0171

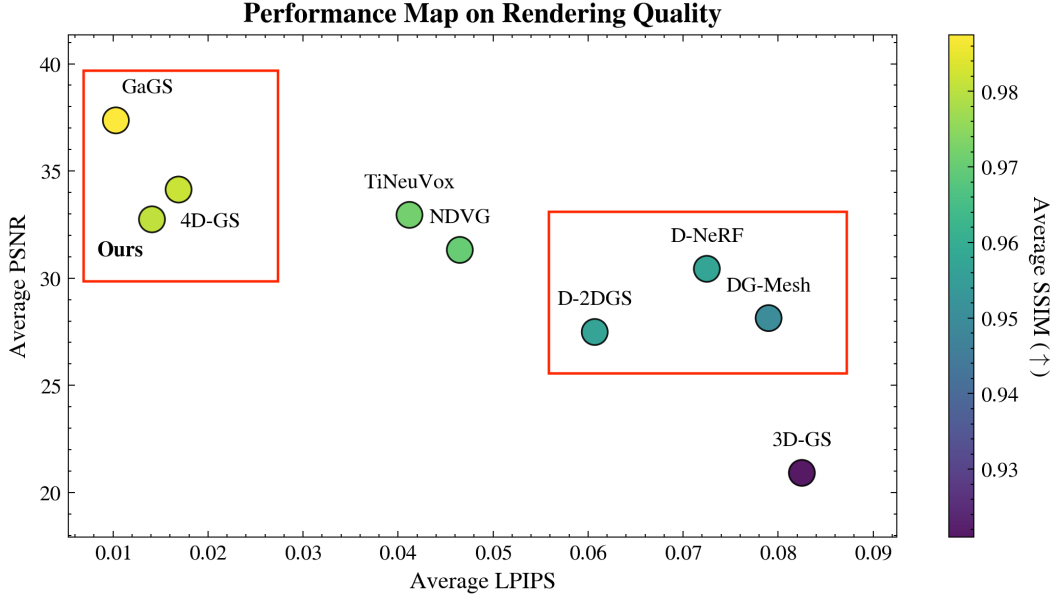


Figure 8: **Average performance map on the D-NeRF dataset.** This scatter plot visualizes the quantitative results averaged over all eight scenes. The x-axis and y-axis represent the average LPIPS and PSNR, respectively, while the marker color indicates the average SSIM score. Our method is highlighted in **bold**.

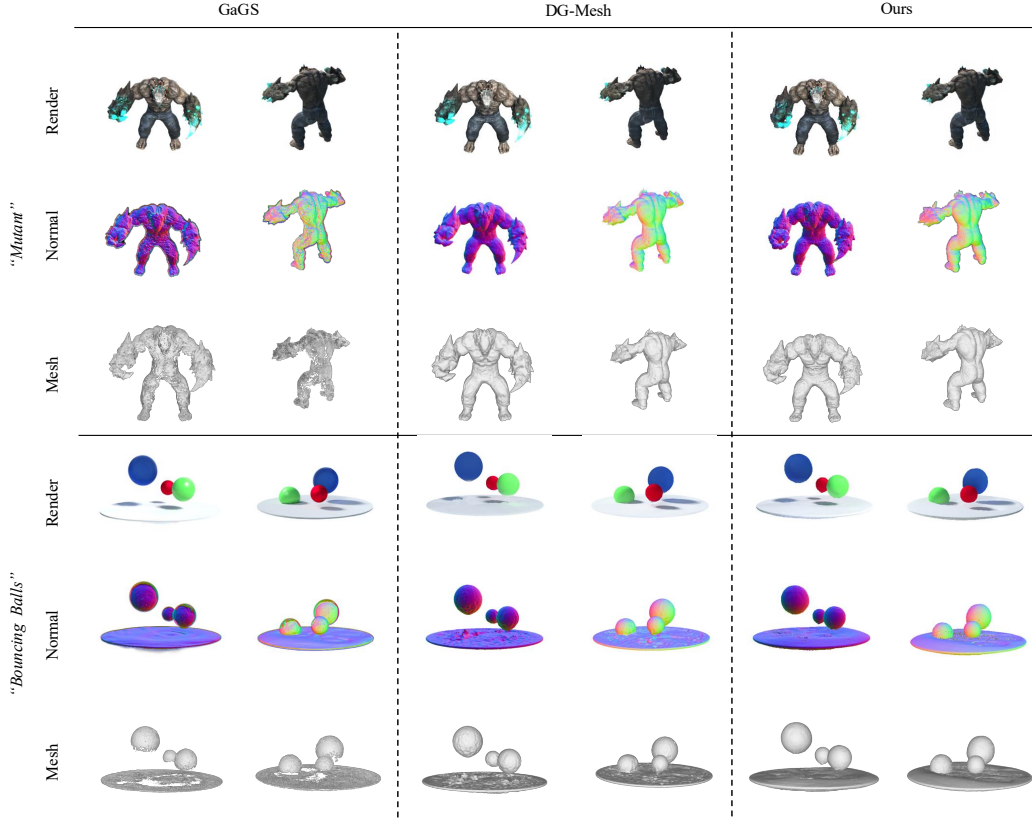


Figure 9: **Additional qualitative comparison results on diverse D-NeRF scenes.** We show the rendered image, normal map image, and mesh surface image results of GaGS [S8], DG-Mesh [S7], and DySurface (Ours). Please zoom to check the details.

Table 9: **Real-world dataset experiment on DyCheck [S11].**

Method	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
D-2DGS	21.91	0.8769	0.1546
DG-Mesh	22.06	0.8705	0.1858
Ours	24.45	0.8871	0.1309

in Tab. 9. DySurface outperforms baseline methods in photometric quality under unconstrained camera trajectories.

C.4 Results on Comprehensive Frame Test

The D-NeRF [36] benchmark is a synthetic dataset generated from dynamic 3D assets. However, the official release provides only rendered image sequences and does not include ground-truth dynamic meshes for geometric evaluation. To establish an accurate geometry benchmark consistent with the motion trajectories used in D-NeRF, we curated the original 3D source assets for *Lego*, *T-Rex*, and *HellWarrior*, and matched their corresponding keyframe animation tracks. Using Blender, we exported the ground-truth deformed meshes over 60 consecutive timestamps covering the full motion interval, $t \in [0, 1]$.

We further conduct a dense-frame geometry evaluation on all available frames of *Lego*, *T-Rex*, and *Hell Warrior*. We report volumetric IoU (vIoU) and Chamfer Distance (CD) for 3D geometric accuracy, together with Depth Mean Absolute Error (Depth MAE) and depth accuracy within 5% of the object scale (D-Acc@5%) under the test view cameras. We additionally provide qualitative depth

Table 10: **Dense geometry evaluation over 60 consecutive timestamps.** vIoU and CD evaluate 3D geometric accuracy, while Depth MAE and D-Acc@5% evaluate rendered depth accuracy.

Scene	Method	vIoU $\uparrow$	CD $\downarrow$	DepthMAE $\downarrow$	D-Acc@5% $\uparrow$
Lego	DG-Mesh	0.2173	0.0731	0.0470	93.09
	D-2DGS	0.1643	0.0439	0.0393	95.74
	Ours	0.2263	0.0439	0.0384	95.69
T-Rex	DG-Mesh	0.5549	0.0538	0.0364	94.34
	D-2DGS	0.4679	0.0565	0.0387	94.31
	Ours	0.5608	0.0580	0.0399	94.45
HellWarrior	DG-Mesh	0.2971	0.0298	0.0225	97.92
	D-2DGS	0.2983	0.0272	0.0255	96.95
	Ours	0.2959	0.0240	0.0250	97.59
Average	DG-Mesh	0.3564	0.0522	0.0350	95.11
	D-2DGS	0.3102	0.0425	0.0345	95.66
	Ours	0.3610	0.0420	0.0344	95.91

and normal renderings to visualize surface accuracy throughout the sequences. As shown in Tab. 10, our method achieves the best average performance across all four metrics, demonstrating accurate geometry reconstruction over dense temporal sequences. Fig. 10 shows the qualitative result of depth and normal renderings.

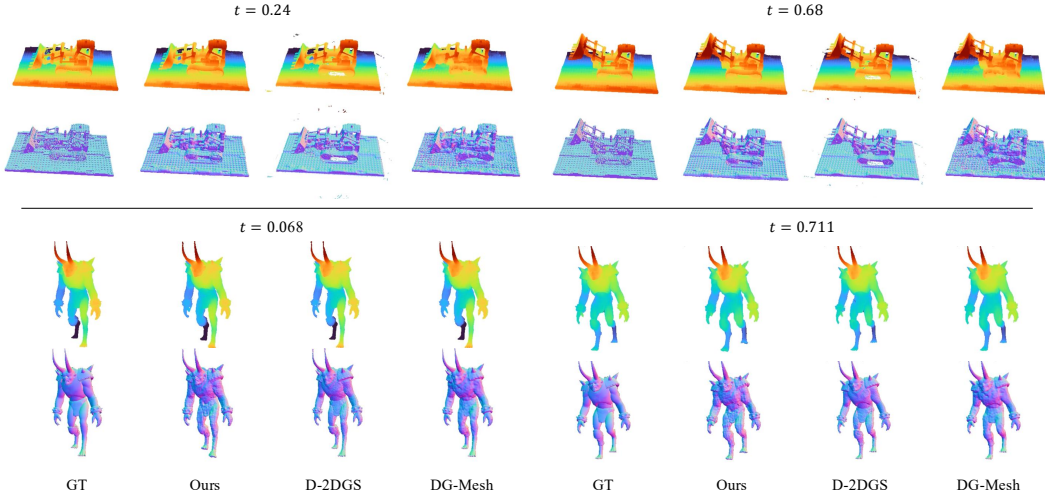


Figure 10: **Qualitative results of the dense frame test.** We visualize the depth and normal-rendered image at randomly selected times, including ground truth.

C.5 Effect of Gaussian Representation: 3DGS vs. 2DGS

To investigate the effect of the Gaussian representation, we replace the 3DGS primitives in our framework with 2DGS [17], while keeping the remaining pipeline unchanged. As shown in Table 11, the 3DGS-based configuration achieves better average performance in both geometric reconstruction and rendering quality. In particular, it substantially improves Chamfer Distance (CD), while maintaining comparable or better photometric quality across the evaluated scenes.

Although 2DGS provides an explicit surface-oriented representation, its primitives are constrained to planar surfels. Under continuous non-rigid deformation, we observed optimization instability and occasional primitive flipping, which can degrade geometric reconstruction. In contrast, 3DGS provides full 3D covariance and greater flexibility for modeling complex deformations. Since surface

Table 11: **Impact of Gaussian representation on the D-NeRF benchmark.** We compare our default 3DGS-based model with a variant using 2DGS primitives.

Scene	Method	vIoU $\uparrow$	CD $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
Lego	Ours-2DGS	0.2943	0.0315	25.09	0.9374	0.0327
	Ours	0.2988	0.0133	25.40	0.9439	0.0245
T-Rex	Ours-2DGS	0.3861	0.0286	36.92	0.9933	0.0072
	Ours	0.5783	0.0050	36.75	0.9932	0.0069
HellWarrior	Ours-2DGS	0.3057	0.0193	31.09	0.9821	0.0195
	Ours	0.3024	0.0124	31.14	0.9835	0.0169
Average	Ours-2DGS	0.3287	0.0265	31.03	0.9709	0.0198
	Ours	0.3932	0.0102	31.10	0.9735	0.0161

regularization is already enforced by our implicit SDF branch, 3DGS provides a more suitable explicit representation for dynamic tracking in our framework.

D Runtime specification

The Gaussian Splatting branch is trained for 2 hours to capture the initial dynamic topology. Subsequently, the implicit VoxGS-DSDF branch is optimized for another 2 hours to build the continuous SDF field. Finally, the joint mesh refining stage requires approximately 1 hour to converge and output the final temporally consistent meshes.

E Limitation and Future Work

While DySurface effectively extracts high-fidelity meshes from dynamic scenes, there exist limitations regarding extreme topological transformations. The Stage 2 implicit SDF presents a clean surface; minor artifacts arise during Stage 3 mesh refinement. Fitting fine-grained photometric losses during non-rigid dynamics can induce minor vertex noise and normal shifts. Increasing Laplacian regularization ($\mathcal{L}_{lap}$) in Stage 3 suppresses these artifacts; however, this may reduce fine-grained geometric details.

In our framework, we define a specific time step t as the canonical space; however, this single canonical space might not contain sufficient context of the object’s surface that undergoes severe fragmentation or topology changes. Extending our approach to effectively model topology-breaking dynamics, such as fracturing or fluid splashing, remains an important direction for future research.

Additionally, while our method excels on 360-degree captures with clear foreground separation, unconstrained in-the-wild videos with severe single-view occlusions receive insufficient photometric supervision, causing localized surface over-smoothing.

F Discussion of Related Dynamic Surface Reconstruction Methods

Comparison with DGNS. DGNS [S9] also combines deformable 3D Gaussian Splatting with a neural implicit surface representation. However, its formulation does not explicitly address the structural discrepancy between the forward deformation of Gaussian primitives and the backward mapping required for volumetric implicit rendering. In contrast, DySurface explicitly couples these two deformation directions through KNN-based RayQuery matching over a dynamically voxelized Gaussian representation, together with the SDF-GS anchoring and cycle-consistency objectives. This design directly associates the dynamic Gaussian geometry with the implicit SDF queries and encourages bidirectional alignment between the explicit and implicit representations. In the qualitative results reported by DGNS, the reconstructed surfaces tend to appear smoother with limited high-frequency geometric details, whereas DySurface preserves finer surface structures.

Comparison with DynaSurfGS. DynaSurfGS [S10] reconstructs dynamic surfaces using planar Gaussian surfels based on a 2DGS [17] representation. Such surface-oriented primitives provide an explicit geometric representation, but their planar parameterization can be restrictive under complex non-rigid deformation. DySurface instead adopts deformable 3D Gaussian primitives, which provide full 3D covariance for modeling dynamic motion, while delegating continuous surface regularization to a multi-resolution hash-grid SDF. The resulting implicit surface is subsequently converted into a canonical mesh, to which the learned deformation field is transferred for temporal articulation. Qualitatively, the meshes reported by DynaSurfGS appear relatively smooth and contain fewer fine-scale surface details, while DySurface better preserves high-frequency geometric structures.

G Statistical Significance

To ensure the reliability and robustness of our quantitative evaluation, we report the statistical significance of the results. For each experimental setting, we independently perform the evaluation 10 times. All values reported in the quantitative tables represent the average (mean) across these 10 runs, accompanied by their corresponding standard deviations ($\pm$).

Table 12: **Quantitative results of statistical significance.** All reported values represent the mean and standard deviation ($\pm$) computed over 10 independent evaluation runs.

Methods	vIoU ($\uparrow$)	CD ($\downarrow$)	PSNR ($\uparrow$)	SSIM ($\uparrow$)	LPIPS ($\downarrow$)
GaGS	0.2422 $\pm$ 0.0031	0.0146 $\pm$ 0.0006	32.24 $\pm$ 0.04	0.9753 $\pm$ 0.0007	0.0181 $\pm$ 0.0002
DG-Mesh	0.3814 $\pm$ 0.0009	0.0137 $\pm$ 0.0002	25.23 $\pm$ 0.04	0.9186 $\pm$ 0.0008	0.1027 $\pm$ 0.0004
D-2DGS	0.3426 $\pm$ 0.0021	0.0111 $\pm$ 0.0004	25.82 $\pm$ 0.05	0.9374 $\pm$ 0.0006	0.0681 $\pm$ 0.0003
Ours	0.3932 $\pm$ 0.0005	0.0102 $\pm$ 0.0002	31.09 $\pm$ 0.04	0.9735 $\pm$ 0.0016	0.0161 $\pm$ 0.0004

H Application Example

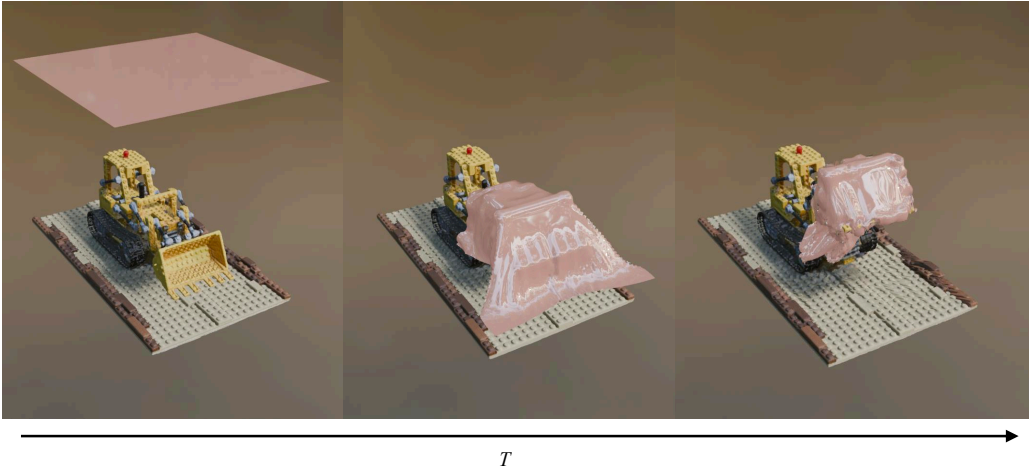


Figure 11: **Illustration of a DySurface application.** We demonstrate an example downstream application enabled by our reconstructed dynamic surfaces. The reconstructed excavator mesh sequence serves as an animated kinematic collision boundary, while a separately initialized cloth is simulated as a deformable body interacting with it.

Mesh remains the predominant representation supported by physics simulators and rendering engines. We illustrate a downstream application of DySurface for physics-based simulation in Fig. 11. Since DySurface reconstructs temporally consistent dynamic meshes from image observations, the recovered geometry can be directly integrated into standard simulation engines without manual retopology. Specifically, we import the reconstructed mesh sequence into the simulator as an animated kinematic collision boundary, where vertex positions are driven frame-by-frame by our reconstruction. We then

drop a secondary deformable object (a cloth in Fig. 11) with assigned material parameters (density, stiffness, damping), and run a forward simulation under gravity. The cloth deforms and responds to collisions with the moving reconstructed surface, demonstrating that DySurface is not only effective for high-fidelity reconstruction but also practical for geometry-aware downstream tasks requiring simulation-ready dynamic surfaces.

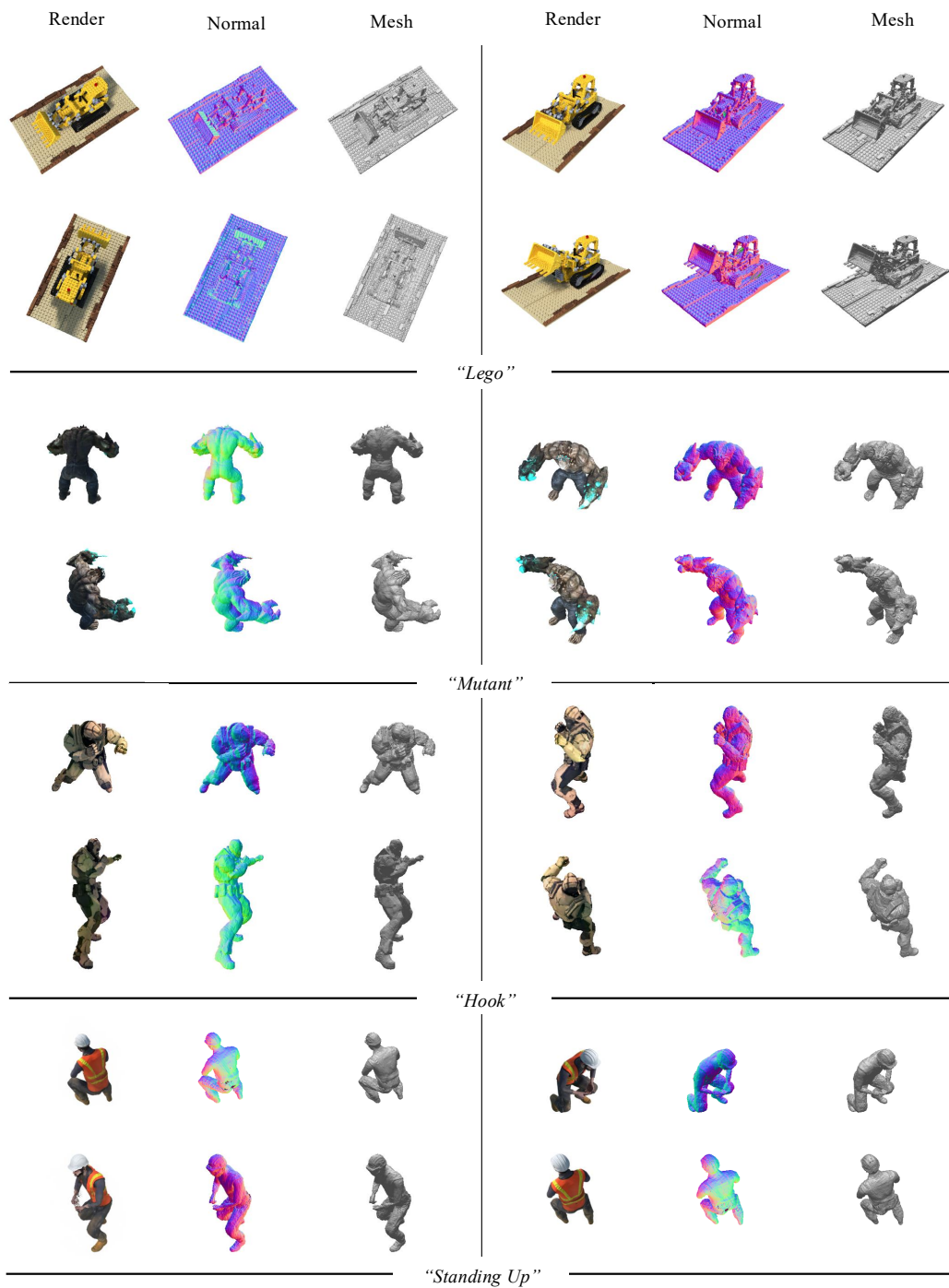


Figure 12: **Additional qualitative results of diverse D-NeRF scenes.** We show the rendered image, normal maps, and mesh surfaces results of DySurface. Please zoom to check the details.

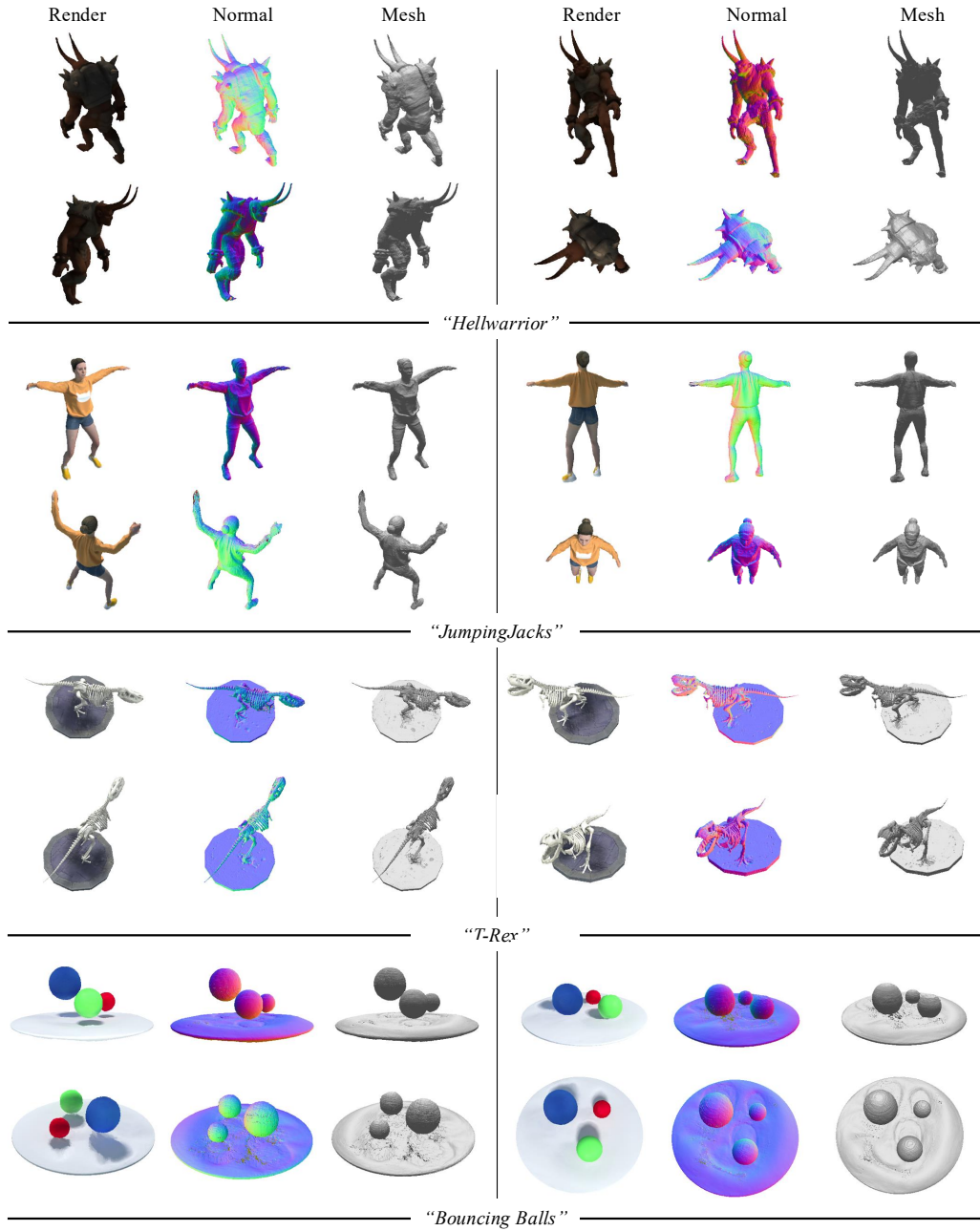


Figure 13: **Additional qualitative results of diverse D-NeRF scenes.** We show the rendered image, normal maps, and mesh surfaces results of DySurface. Please zoom to check the details.

References

- [S1] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D Gaussian splatting for real-time radiance field rendering. In SIGGRAPH, 2023.
- [S2] Albert Pumarola, Enric Corona, Gerard Pons-Moll, and Francesc Moreno-Noguer. D-NeRF: Neural radiance fields for dynamic scenes. In CVPR, 2021.
- [S3] Jiemin Fang, Taoran Yi, Xinggang Wang, Lingxi Xie, Xiaopeng Zhang, Wenyu Liu, Matthias Nießner, and Qi Tian. Fast dynamic radiance fields with time-aware neural voxels. In SIGGRAPH Asia, 2022.
- [S4] Xiang Guo, Guanying Chen, Yuchao Dai, Xiaoqing Ye, Jiadai Sun, Xiao Tan, and Errui Ding. Neural deformable voxel grid for fast optimization of dynamic view synthesis. In ACCV, 2022.
- [S5] Jingbo Zhang, Xiaoyu Li, Ziyu Wan, Can Wang, and Jing Liao. FDeRF: Few-shot dynamic neural radiance fields for face reconstruction and expression editing. In SIGGRAPH Asia, 2022.
- [S6] Guanjun Wu, Taoran Yi, Jiemin Fang, Lingxi Xie, Xiaopeng Zhang, Wei Wei, Wenyu Liu, Qi Tian, and Xinggang Wang. 4D Gaussian splatting for real-time dynamic scene rendering. In CVPR, 2024.
- [S7] Isabella Liu, Hao Su, and Xiaolong Wang. Dynamic Gaussians Mesh: Consistent Mesh Reconstruction from Monocular Videos. In ICLR, 2025.
- [S8] Zhiwen Lu, Xiang Guo, Le Hui, Tianrui Chen, Min Yang, Xiao Tang, Feng Zhu, and Yuchao Dai. 3D geometry-aware deformable Gaussian splatting for dynamic view synthesis. In CVPR, 2024.
- [S9] Xuesong Li, Jinguang Tong, Jie Hong, Vivien Rolland, and Lars Petersson. DGNS: Deformable Gaussian Splatting and Dynamic Neural Surface for Monocular Dynamic 3D Reconstruction. In ACM MM, 2025.
- [S10] Weiwei Cai, Weicai Ye, Peng Ye, Tong He, and Tao Chen. DynaSurfGS: Dynamic Surface Reconstruction with Planar-based Gaussian Splatting. arXiv preprint arXiv:2408.13972, 2024.
- [S11] Hang Gao, Ruilong Li, Shubham Tulsiani, Bryan Russell, and Angjoo Kanazawa. Monocular dynamic view synthesis: A reality check. In NeurIPS, 2022.